

LeNet-5 in Mojo Without Libraries Review

What is Mojo?

Compiled, memory-safe, high-performance, MLIR-based(*), heterogenous-compute native, SIMD native, machine-learning oriented. 1.0 vs later releases. Also, MAX, Mammoth, etc. Generally, I like it a good amount. It doesn't make me feel as "dialed-in" as Zig, or as fast as Python for rapid prototyping, but it fills an in-between space that I'm learning to appreciate. I imagine this is how people get roped into languages like C++ - in an attempt to abstract away some behaviors, you save a lot of boilerplate and thus can iterate faster, but sometimes those abstractions come back to bite you (comptime unrolling of compiler-synthesized InlineArray copies, for example). Once more support gets added for networking, async, some QoL like enums, I would start to have a hard time wanting to go back to anything else.

Python

The glue language for modern machine learning and many other domains. Performance isn't great, but it's recognizable and comfortable for a lot of people, especially in machine learning / data science. The syntax, where possible, is meant to stay as close to the Python standard as possible.

C++

A lot of Python's performance critical areas have to reach into C++ - Mojo attempts to solve this "two-language" problem. The founder, Chris Lattner, obviously is very familiar with C++, what it can do and why, as he's the guy effectively behind the ubiquitous clang compiler. C++ isn't known to be a particularly ergonomic or consistent language, but it's an "worse is better" language that people both complain about and use constantly.

Zig

Zig came along to improve the experience of C - less undefined behavior, modern features like Result / Option. One of the main reasons I'm mentioning it is for its explicit use of "comptime" as a macro-adjacent system to avoid the messiness of generics and preprocessor macros like we've seen in the C family of languages. It allows a programmer to do some work at compile time to do things like write clean, generic functions, to precalculate values, to optimize things like tensor shapes and boundary checks, along with opening up easy explicit paths for optimizing a binary for a particular set of hardware and so on. We'll see this in the "parameters vs arguments" language when we get to actual code.

Rust

Infamous for its borrow checker in a novel look at memory-safety, Mojo has its "origins" and ASAP destruction team to manage memory.

CUDA / HIP / Metal

My experience for GPU programming is from using CUDA with C99 - tons of boilerplate, very limited APIs and tools. Everything I know is raw pointers and runtime constants. Mojo allows for the same structs and methods you write for "hosts" (CPUs) to often just work on "devices" (GPUs) - and many of them, not just whatever vendor you're writing for. I don't need to write an activation function for both spaces, one works on both. I can pass LayoutTensors and use normal indexing "my_tensor[row, col]" instead of N-dimensional folding and flattening. While I didn't opt-in for this project, you can write parallelizable code in a functional way that allows you to dispatch to a host or device depending on runtime control flow and things just work - I'm hoping to explore this more when I can. This is one of the HUGE areas for me that attracts me to the language.

What is LeNet-5?

This is a famous “Convolutional Neural Network” by now-famous Yann LeCun et al. that was trained to convert images of hand-written digits to numbers predicting what they are. The model works by having the following structure: input -> convolution -> maxpool -> convolution -> maxpool -> “convolution” -> linear -> output. The model starts with random weights and is trained on labeled data using “backpropagation” until accuracy and “loss” are where we want. Since there are so many resources on what CNNs are, I’ll leave it up to the reader to decide how much they want to dig into this. Regardless, all you really *need* to know is that there’s a decent amount of matrix operations and we want high performance.

This implementation and all previous versions I’ve done were without libraries. Everything was written from scratch - models, weights, kernels, logging, training loops, everything without libraries. No PyTorch `nn.linear()`, no autograd, no simple “swap optimizers”.

Why LeNet-5?

Dr. Pan and Dr. Xie and Dr. Prasad

When finishing my undergrad (finally after many years of sober bartending), I took a few courses that gave me good exposure to this model. Dr. Xie had me use it for medical research combined with compressed sensing on a compute-constrained device (a pacemaker trying to detect heart failures); this model is small and pretty accurate for ECG signals. This continued with a “AI Hardware and Programming” course from Dr. Pan over in the EE department that taught a lot about cache management, hardware acceleration, and methods for compressing models (we implemented pruning, quantization, and GPU kernels for a Jetson Nano) for a version of LeNet-5 provided to us in C, available here for reference: <https://www.startpage.com>. For our final project, we needed this to run on a Jetson Nano (small NVidia device) along with a MSP430 embedded system that’s rather resource constrained - necessitating quantization and / or pruning to even fit the weights on device.

Lastly, Dr. Prasad really helped deepen my knowledge in the space of machine learning with a Parallel Programming course (cross-listed with undergrad and grad) and High Performance Machine Learning, a grad course with a few seats open for some of his Parallel Programming students. Here, we covered mostly the opposite end of the spectrum from constrained compute - large scale computation. Butterfly networks and hypercubes, MPI, OpenMP / Pthreads, CUDA. Model and data parallelism, tensor offloading, federated learning, all of that good stuff, with plenty of time spent on fundamentals of how neural networks work. LeNet-5 was a presentation I did for this course. My final project was about Modular and Mojo’s attempts to build a “from first principles” stack of software to address modern needs in this space: great compilers, plenty of hardware support, easier integration into distributed layers, etc. (This needs a lot of work to explain). tl;dr - I know this model pretty intimately. It’s provided a great platform for a lot of learning exercises over the course of more than a year (on and off nights and weekends), and doing it in C + CUDA and now Mojo (once already) means I’m able to constantly improve and grow with better patterns, engineering, code quality, performance, and general levels of polish.

Old Versions (Pros and Cons)

The original C version is CPU-only, and very simple, but it performs very well all things considered. Read it for yourself and check the benchmark section. A big thing to note that we’ll come back to is: parallelization, and stack-allocation of the model itself. OpenMP is very simple to use for parallelizing single image passes, and the way that we can simply define a struct with many layers like `int weight0_1[1][6][5][5]; int weight_2_3[6][16][5][5]` is really “dumb” but efficient: all memory resides next to each other contiguously. We’ll revisit this concept as well.

My first CUDA version made a LOT of mistakes - namely, not batching things and synchronizing too much. Under a deadline crunch for this final project submission, I did just a very dumb and fast conversion of the CPU path over to GPU. This is not ideal! Again we'll touch on why this is so awful later.

My first Mojo version had a few key failures: I basically did a 1:1 translation of the C version to start. The main issue here is organization - object oriented programming done correctly can add a ton of stability and sanity to a project. Additionally, this was my first stab at the language and I was still new to GPU programming, so the CPU path didn't have SIMD or parallelization, and the GPU path was just not great, repeating many failures from before (namely synchronizing too much). Performance was also bottlenecked due to small batch sizes based on some decisions I'd made about argument passing to and from kernels. While it remained similar on GPU to PyTorch by metrics (that I don't trust and need to double check), I was happy to get it running at least. The language also didn't quite have all the modern features I now rely on: simple things from appending to files, to reflection, to better compiletime assertions. I also wasn't remotely aware of all of the features it has, like compiletime use of `env defines` to produce lean and mean binaries.

Goals For This Project

- beat PyTorch etc without stipulations
- try new GPU features (streams, warp / block primitives, etc.)
- polish!
- benchmarking, **especially** against Metal and AMD GPUs
- more "features" - useful traits, allocators, activation functions, cli parsing
- learn better how PyTorch, JAX, ONNXRuntime, etc work under the hood

This project needed a lot to really warrant a "2.0" release. I wanted to push it to its limits. Here's what I passed on:

- that last theoretical `unk%` gains in performance on GPU. I imagine up to 20% is possible, mostly in `conv3` and `matmul`. Kernel fusion would help a lot as well.
- CPU performance (batching etc would require some larger changes to memory layouts and the like that I don't think would teach me as much)
- "CUDA Graphs" etc, there are ways to build and call these from Mojo (external calls to C APIs to capture and replay streams) but until they're natively in the `stdlib`, I'm not going to worry
- optimizers - [batched] gradient descent is all you get for this one. If I finish my Mojo LLM, we'll have an implementation to talk about (dangling idea).

Technical Details

Allocators

Bump Arena Allocation

A perhaps dry topic, but one that gave a nearly immediate 20% performance improvement that ties back into the "dumb C" version. A quick reminder, when we call to allocate memory on the heap (`malloc` in C, `alloc` in Mojo), the default allocator works with the OS to find a chunk of memory it can use somewhere in the heap. This is general purpose, but provides no guarantee that the memory is anywhere near itself. That means if we want to, say, allocate a few MNIST images or feature buffers, depending on how it's done, they could be very far from each other. When we need to access some number of those, we're having to jump all over the place in memory to find them.

Instead, we introduce a very simple allocator, the arena allocator. Since we know batch sizes (or max batch size) at many points, we can allocate the total space needed all at once, and then put all of our images in there together. This is done by getting our arena, setting an offset to zero to start, then “allocating” from the arena by simply moving the offset by the size of the needed allocation and returning the base address plus that offset as a new pointer for the new object. This can be written very simply (and allows for setting your own error handling, resizing, logging, memset the whole space to zeros between passes, easily save all members to file, etc) and the performance gains for using this strategy + reusing buffer space improved performance on CPU by about 20% immediately without other changes. This strategy isn’t always perfect - if you get into the weeds, you can run into issues where you saturate one of your DRAM “ranks” and the other sits idle, so splitting the data could introduce overall throughput, but for this small amount of data and simple project, getting away from the general malloc strategy is clearly worth it. I’m hoping that Allocators release with Mojo 1.0.

GPU Pinned Memory

GPU allocation works differently. We generally think of the GPU as a disconnected device (in the asynchronous sense) and have to be very explicit about communications. The CPU “host” tells the GPU “device” that we need memory. We send that call, then have to transfer memory from host to device (and back when needed). These synchronization costs are very high and we want to avoid data movement as much as possible and be smart about it, too. We’ll get into more of this later, but generally if you use “normal” host-side allocations and want to send some data, you’re forced to do it all synchronously which means incurring costs that you might not need to, especially if you want to overlap data transfers with compute (which want to do and we will talk about shortly). Instead, to enable asynchronous transfers, we use specialized host allocators that hand us “pinned” memory. The operating system treats this differently than most allocations, with a big difference being that we know this allocation won’t get “paged” and send out from RAM to disk which is very costly if we need to go and fetch it. The OS also knows that other devices might want to communicate with it, and it can help protect that relationship and facilitate it. It also can basically guarantee that the transfers will be asynchronous, which means fewer interruptions, and thus more time for actual work to get done. This was about a 15-20% improvement as well for GPU! We used an arena for GPU as well, but benchmarks haven’t quite been done for that yet.

GPU Streams

GPUs, to reiterate, are “dumb” devices that we have to synchronize and control from the host. They also have dedicated hardware for compute, transferring to host, and from host. That means that if we want greater total utilization of GPU, we need to ensure all of these are working at the same time. If we think sequentially, to run an inference batch, we have to transfer the image buffers to the device, once that’s done and confirmed we can queue all of our kernels, and finally once the outputs are produced, we can finally request to have those sent back to the host. For the next batch, we go back and start. H2D -> compute -> D2H. H2D -> compute -> D2H. No overlap - sequential work in that order.

What we can do to help is to open up another “stream” to the GPU that provides its own scheduling and communication. The stream is what we use to enqueue memory transfers, enqueue kernels, and the like. Streams can synchronize and make calls independently, meaning that while one is computing (for example), the other can be transferring. Now, we can pipeline our workloads so that our three main tasks (H2D, compute, D2H) can overlap. The change isn’t that large, either - if we have N streams, we have a “prologue” that sets up each stream (with its own buffer pointers, pinned staging on host, etc) and begins uploading a batch of images for each. This is why I have to report “effective batch size” compared to other frameworks; if, for instance, we have 4 streams with 50

images handled by each, we have a batch size of 200 running at a given time at full workload. Then, we start looping through our data, slicing out each batch (let's say we have "B" batches to run), and sending it to a stream to handle. Once we've begun stream N, we start looking for the results of batch "B - N" in its StreamSlot, and gathering those results. At the end of all B, we have to do a final gather of the last N batches in an "epilogue" or "drain" loop.

Multi-streams may or may not always speed things up. For most balanced setups, common wisdom appears to say that having more than 3 streams or so probably won't gain much benefit - at some point either H2D, compute, or D2H is your bottleneck, and more streams won't help that. Depending on my kernels, sometimes I saw exactly this behavior. In my current compute pipeline, I'm able to keep seeing gains beyond just 3 streams. The current default is 5 on my device.

Custom Kernels

If you haven't done GPU programming, you should! It's fun (if you like games like Factorio, you'll be having a blast)! Modular has great GPU programming puzzles that I would recommend to anybody. GPU programming is still something I'm growing in, but there's a few main ideas to focus on when writing kernels:

- Compute is handed out to "grids" of "blocks", each has a large number of "threads". Typically, on CUDA-based models, you can have up to 1024 threads per block; threads are physically implemented in fundamental hardware units called "warps" that are 32 threads / lanes (64 is typical for AMD).
- use profiling to identify hotspots. Right now, only CUDA / nsysprofile is supported out of the box, but surely AMD and Metal solutions will follow.
- if your army of threads are going to be reaching for the same data over and over, pull them into shared memory that's more local than device global memory. You have to manage memory a little more hands on than most CPU systems where the OS and hardware handle a lot of very intelligent and good caching for you.
- be mindful of how many blocks you call and how those map to SMs. Sometimes you can pull off having a ton of blocks doing a little bit of work each, and that saturates the GPU nicely. Sometimes it's better to have fewer blocks doing more work. It depends on what else the GPU is doing, the scheduler, synchronization costs, etc. This can possibly explain some of the num_streams results we saw.
- there are some cool tools we can use: let's say we want to load a chunk of weights into a block's shared memory. A naive way is to have each local thread go to global memory, pull that into a local register, and then send it to shared memory. Instead, we can use functions like "copy_dram_to_sram_async()" to skip having to use the local registers; this is also asynchronous which could be useful in some applications (pull in new data while we're computing, etc). there are also things like block-wide or warp-wide "reductions" that can, for instance, sum up all values in that unit efficiently by using specialized hardware / driver features.
- "tiling" is a concept you'll want to learn; the TileTensor is often replacing the LayoutTensor in a lot of stdlib code because it facilitates this pattern so well. It's even a useful convention for CPU operations.
- set things up so you can easily tune for *your* hardware
- graphs, if available to you, can help even more for static compute pipelines by "capturing a stream" and being able to replay it as one unit instead of having to call each kernel manually. This can eliminate kernel call overhead
- just keep practicing. There are sites like leetgpu and tensara that offer LeetCode-like challenges for GPU and include free GPU usage.

Fixing Old Mistakes

- way too much GPU synchronization. I was synchronizing for each results buffer. That's 10,000 syncs alone just for getting those results (and the D2H transfers were much larger - (10 * 4B) (fp32) per image instead of a single 1B label guess. final tallies are still done on CPU, but argMax isn't any longer).
- make sure your benchmarks are apples-to-apples, I was actually making things harder on myself by not allowing for warmup runs, by timing normalization costs, by timing some setup costs, etc, all things that made my performance look worse than it was. This also made it harder to understand what my actual problems were.
- using shared memory when I didn't need to. I figured it "probably couldn't hurt too much" and just did it for all kernels; but those small inefficiencies add up.
- not reusing "scratch pads". Don't keep reallocating new buffers, just zero out the old ones or overwrite them. This means more work in earlier calls in the pipelines that make things a little less obvious and logical to read, but make a BIG difference.
- SIMD is *the* fundamental unit for Mojo's math datatypes, but as far as I'm aware, there's very little auto-vectorization at the moment that more mature compilers (like clang) are able to pull off. Learning how to use SIMD is pretty darn easy in Mojo and can speed up a lot of operations. By also being explicit and much more ergonomic, you can feel a lot more certain about what your code is likely to actually compile to.
- CPU parallelization. Not sure why I hadn't done this in the last version, but hey, it's also very easy to do (though not a very comprehensive system for Mojo *yet*). This can be used in a number of ways; my simple implementation was to just use multiple threads per batch so each thread processes an image. If you have 12 logical cores on your CPU, you could expect to get up to a 12x performance increase (though in practice, less). A better pattern would be to gather each image into a more cohesive batch, and write batched operations. That way, we can reuse weights more effectively across threads instead of them potentially fighting for resources. This was enough of a rework that I felt I was already comfortable with that I didn't go ahead and implement it, but it should be doable in a weekend's playtime. As part of this, I'd also change memory layouts of my weights to [OC][IC][etc] from [IC][OC][etc].

Actual Implementation (Tutorial-Lite)

Quick Mojo Tutorial

It looks largely Pythonic, and I'll assume the general audience understands things like data types, functions, etc. Here are some basics to help you read things:

```
var x: Int = 42 # '# for comments'. everything has a static, compile-time known type.
it can be inferred, and the use of 'var' is essentially optional // link to the
explanation of this
x = 42 # this is still perfectly valid
```

```
def main(): # entrypoint
def main() raises: # if something can raise an error, it must be explicitly stated
with "raises"
```

Let's look at a big feature, parameterization. Mojo splits compile-time "parameters" from runtime "arguments". The former are passed or inferred first with [], the latter as "normal" within (). Let's look at a quick example:

```
def multiply(x: Float64, y: Float64) -> Float64:
    return x * y
```

```
# that only works for one dtype: Float64, and the language has moved away from
# implicit conversions, so that matters even more. Let's make it properly generic:
```

```
def multiply[T: DType](x: T, y: T) -> T:
  return x * y
```

```
# now we can do
```

```
...
x = multiply(1, 4)
x = multiply(3.14, -1.96)
```

In the first call, we are passing Ints. The compiler will infer ‘T’ for us to be Int, and compile the function for that type. When we call it again with floats (default is Float64), the compiler can infer that and build the assembly correctly for us again. This becomes rather powerful, and the explicit nature of it gives a lot of peace of mind. You can avoid a whole class of silliness brought about by generics, preprocessor macros, and the like. It can lead to some... rather *verbose* looking function signatures, but you end up saving a lot of headaches in debugging, and gain much more predictability. A cool place we can use this is for choosing our activation functions; by selecting them at compile-time, we can easily pass them around and have much stronger guarantees that they’ll get properly inlined (if we don’t just tell it to explicitly, which I’m also doing). More optimizations can be done by the compiler if it’s given more firm information from the get-go. Another common place I like to use these is for branching logic - we can set flags at compile time, and use `comptime if _` to pick the right branch and not even have to compile the others that we know cannot ever be reached. This can be used for choosing to take a hard-exit path on failure within a function, provide bounds on indexing to avoid having to check bounds constantly (great for performance), and so many other things.

CLI Parsing

A basic tool, but we’ll want to be able to test and set a number of conditions while we run things, and this is a simple enough showcase of some language features and pain points. We have the following pattern:

```
def get(mut self, tag: String, default: Int, desc: String) -> Int:
def get(mut self, tag: String, default: Float64, desc: String) -> Float64:
```

Overloaded methods. I was hoping to be able to get this parameterized on `T: AnyType` from a set of `T` that allows for conversion from Strings and raises on failure, but can’t find a way to do that.

Logger

Let’s start with something basic: logging. We’ll want this eventually to be able to track training and testing runs and the like. This is slightly overbuilt, but I did so early on to learn how to use traits.

MNIST Data (image.mojo && dataloader.mojo)

Let’s get started. We need data! MNIST is now an infamous “Hello World” in the neural network space - 28x28 greyscale (single-channel) images (one byte per pixel) of hand-written digits with labels. These come as binary files (with magic number headers) of raw bytes. See `data/`.

Images

Our model will accept “Images”, so let’s plan that out, and then load MNIST into this format.

```
// how would i best show the filename and maybe link it
struct Image(implicitlyCopyable): # this trait makes it simple to copy this around
  # let's define the "raw" representation, 28 x 28 bytes for MNIST
```

```

comptime PixelLayout = Layout.row_major(IMAGE_SIZE, IMAGE_SIZE)
comptime PixelTensor = LayoutTensor[
    DType.uint8, Self.PixelLayout, MutAnyOrigin
] # raw pixels

# this is what the model will want: (normalized) floats, and padded
comptime DataLayout = Layout.row_major(PADDED_SIZE, PADDED_SIZE)
comptime DataTensor = LayoutTensor[
    ftype, Self.DataLayout, MutAnyOrigin
] # normalized into ftype and padded

var pixels: Self.PixelTensor
var label: UInt8 # digits [0, 9] MNIST, could store as "Int"

```

Notice: the data itself isn't "stored" here. A LayoutTensor is just a fat pointer into memory for the data, plus some information like shapes and strides. You might find that more logical for your implementation, but since we're already working with the MNIST binary files, there's no reason to duplicate things. Just read the file into RAM, and point into it, done. That also gives a performance benefit: contiguous storage of data! One thing we'll have to manage of course, is lifetimes. We are pretty happy to just use "MutAnyOrigin" in our Tensor declarations: this means the data is allowed to be changed and could come from anywhere, so that has to be handled separately. We could (and might in future revisions, though this isn't load-bearing by any means) parameterize each Image on origin, but we'll just rely on the idea that the MNIST repository will be "alive" long enough.

Data Loading

Let's read some files and get some data. Our data repository can (and should) be tightly coupled to the MNIST specification. To set it up, we need to point it to files, decide how and where we'll store those files in memory, and do some error handling. This should look rather Pythonic (and even moreso if I didn't insist on using 'var' everywhere for declarations: you don't have to use 'var' but it gives tighter scoping than not using it (Python declaration gives Pythonic scoping - block level)).

The 'comptime' keyword is exactly what it sounds like: compile-time known constants. I encourage you to read up more on comptime, but they allow us to talk directly to the compiler and that allows many of these things to disappear from the binary and can enable a number of optimizations. Historically, compilers have gotten quite fantastic at figuring out situations like these (unrolling loops that have hard-coded ranges), but being able to "talk" directly to the compiler explicitly with statements and expressions and type aliases provides a lot of simplicity and guarantees.

```

from std.pathlib import Path
struct MNISTDataRepository:
    comptime COUNT_TRAIN = 60000
    comptime COUNT_TEST = 10000

    # some private arenas are omitted from this snippet

    # Lists are easy to work with
    var test_data: List[
        Image
    ] # Image is a label + a (fat LayoutTensor) pointer into the arena for the
pixels
    var train_data: List[Image]

    var data_dir: Path
    var train_image_file: Path
    # test image, train label, test label paths

```

```

def __init__(out self, data_dir: String = "data"):
    # setup paths
    self.data_dir = Path(data_dir)
    self.train_image_file = Path(data_dir + "/train-images-idx3-ubyte")
    # etc

    # call methods like below, one for train, one for test

def _readTrainData(mut self) raises:
    try:
        var data_file = open(self.train_image_file, "r")
        var label_file = open(self.train_label_file, "r")

        _ = data_file.seek(
            16, os.SEEK_SET
        ) # data has a magic header # 2049
        _ = label_file.seek(8, os.SEEK_SET) # labels too # 2051

        comptime size = Image.PixelLayout.size()
        for c in range(Self.COUNT_TRAIN):
            var data_list = data_file.read_bytes(
                size # IMAGE_SIZE * IMAGE_SIZE
            ) # -> List[Byte]

            var temp = label_file.read_bytes(1) -> List[Byte]
            var data_label: UInt8 = temp[0]

            var img = Image(data_list, data_label, self._train_pixels_arena)
            self._train_labels_arena.buffer[c] = data_label
            self.train_data.append(img^)

        data_file.close()
        label_file.close()
    except e:
        raise Error(t"Error with input MNIST test binary files: {e}.")

```

Model

Fundamentally, the model is just a set of weights. Our main tool for this will be the LayoutTensor, which we'll get more comfortable with now. Here's the model definition:

```

struct LeNet5(Movable, ArenaSizable):
    # var arena: Self.Allocator # might not actually be an 'arena' per se, but
    # that's the default
    var allocator_owns_memory: Bool

    # WEIGHTS
    comptime w01_layout = Layout.row_major(
        INPUT, LAYER1, LENGTH_KERNEL, LENGTH_KERNEL
    )
    var weight0_1: LayoutTensor[ftype, Self.w01_layout, MutAnyOrigin]

    # weight2_3
    # weight4_5
    # weight5_6

```

```

# BIASES
comptime b01_layout = Layout.row_major(LAYER1)
var bias0_1: LayoutTensor[ftype, Self.b01_layout, MutAnyOrigin]
# etc

def __init__(out self, mut arena: Some[CPUAllocator]):
    """
    Initialize to all zeros, for training you'll want to randomizeWeights(),
    or for inference, read in from a file. Only biases really need to be set
    to zeroes.
    """
    self.allocator_owns_memory = True # for destructor cleanup

    # weights
    self.weight0_1 = LayoutTensor[ftype, Self.w01_layout, MutAnyOrigin](
        arena.alloc[sftype](comptime (Self.w01_layout.size()))
    ).fill(0.0)

    # the rest of the weights and biases follow the same pattern

```

Let's go through this. The struct definition has two traits: Movable and ArenaSizable. The former should be largely self-explanatory. ArenaSizable is a trait I defined that guarantees presence of a `sizeInBytes()` method that, well, returns the size of the struct in bytes. This is super handy for initializing allocators / regions of memory. Cool. The `LayoutTensor` needs three main things: the datatype (`ftype`, defined as `DType.float32` in `constants.mojo`, which also contains most definitions and layouts etc.), the compile-time-known `Layout` (you can have runtime layouts, but they'll remove some `comptime` magic and are limited to the rank you pass here), and an "origin". They're worth reading about on their own, are similar to Rust's lifetimes, but `MutAnyOrigin` is easy enough to pass around for now. We could (and might) parameterize the whole struct to the underlying data storage to have a clean, clear origin, but the important thing to read here is that "Mut" means we will be allowed to mutate the data stored by the underlying pointer. To initialize, we simply pass in a pointer (`UnsafePointer` is a C-style pointer, just holds an address, nothing fancy about it and indeed unsafe if used incorrectly), or a `Span` (fat pointer with origin information) to the constructor. We're using a homemade allocator to get our needed pointer, but that's less important.

* We end up fixing this by creating a CPU and GPU "Session" that ties all of its members together so the lifetimes *have* to be shared. As far as I understand, origins still have some work on them that needs to be done (and documentation was limited when I started this project), especially around things like iterators, so I picked an "easier" pattern with the Session "trick". Had I not, Mojo's ASAP destruction might invalidate all of my arenas that hold weights before the model is done, which is very double plus ungood. But, at least this way, we know memory stays valid!

CPU

The CPU version is the above, plus some options to load and save from file, random weight initialization, and the like. The forward method is just calling the kernels we need which we'll cover more below. We can show off some SIMD action - during training we'll need to accumulate the results of backpropagation into our weights, so that will mean we need a method to make that happen. Let's take a look:

SIMD Aside

```

@staticmethod # passing "self" won't really help here, so we make it a static
method on the struct

```

```

def _accumHelper[
  x: Layout
](
  accum: LayoutTensor[ftype, x, MutAnyOrigin], # an indication that mutation
will be performed is explicitly needed here both for the compiler, as well as for a
more readable API
  other: LayoutTensor[ftype, x, _], # we don't care what the origin is. the
compiler will silently deal with it for us, nice!
  lr: sftype, # Scalar[ftype] means just a SIMD[ftype, 1], or a single float
(whose precision is defined in ftype, default is float32)
):
  comptime N = x.size() # if the Layout has 1000 elements, .size() returns 1000
  _ = "" # here is a naive implementation. easy enough
  var a = accum.ptr # grab the starting addresses
  var b = other.ptr
  for i in range(N): # we could comptime unroll this, but i avoid that unless
necessary // TODO in this writeup, pain points of Mojo = comptime unrolling being
slow!
    a[i] += (b[i] * lr)
  ""

  # let's vectorize with SIMD
  def vectorize_closure[width: Int](i: Int) {read}:
    var lrs = SIMD[ftype, width](lr)
    var a_nums = accum.ptr.load[width=width](i)
    var b_nums = other.ptr.load[width=width](i)
    var result = a_nums + b_nums * lrs
    accum.ptr.store[width=width](i, result)

  vectorize[nelts](comptime (N), vectorize_closure)

def accumulateFromOther(
  mut self, other: Self, lr: sftype
):
  ""
  For taking in errors / deltas during backward pass with learning rate.
  self.weight0_1 += other.weight0_1 * lr # this simple attempt EXPLODES COMPILE
TIMES
  ""
  Self._accumHelper(self.weight0_1, other.weight0_1, lr)
  Self._accumHelper(self.weight2_3, other.weight2_3, lr)
  ...
  Self._accumHelper(self.bias5_6, other.bias5_6, lr)

```

Alright, lets work through this! In the code is a naive example, loop through the weights by using the underlying pointer and known size, accumulate. We don't need to do fancy indexing and worry about shapes, only size. This is an elementwise operation.

But!- we can do this with SIMD operations to process a wider lane of “nelts” (**n** umber **ele** ments) at once (speedup is more or less theoretically about equal to ‘nelts’). The ‘vectorize’ function is the main thing we need - let's begin by understanding its signature. ‘nelts’ is passed as a compile-time parameter so we know how wide of a SIMD lane to use. Typically this will indeed just be `simd_width_of[dtype]()`. The two runtime arguments are first the number of elements to be processed, and the name of the closure we want to use that has the expected signature `[width: Int](i: Int)`. {read} tells us that the closure is allowed to read local variables and won't need to modify

anything. Closures, at the time of this draft, still need some documentation, so you might just have to “roll with it” for now.

The ‘i’ is the index: for $N // \text{width}$ this means ‘i’ increments by width, and for the last

$$N \% \text{width}$$

elems it will increase by 1. Vectorize automatically creates this “drain” loop behavior where the closure is built with a “width” of nelts and 1. I.e., if we need to run 19 elements with a width of 8, vectorize will call the closure twice with a width of 8, then three times with a width of 1. The code should be otherwise easy to read: build some SIMD vectors, perform some operations on the chunk (these are all element-wise), and then write that chunk. ‘N’ has to be wrapped in `comptime(N)` to tell the compiler that it will have to evaluate this at runtime; I’m not super certain when this is needed or not.

GPU

This gets more complicated - but it will pay off! What’s complicated? Well, for one, we want to be super hip and just be able to pass our entire model to the GPU like a native type. We could just pass the tensors which still offers the benefits, but check this out! I like this! I’m not sure if / how you could do that in C++ / CUDA. This means we can do really simple things like `lenet5_gpu.weight0_1[ic, oc, kw, kh]` and not have to deal with the normal “pass a pointer, do some manual indexing, lots of checks, etc” dance.

Compare the following generic example:

```
__device__ void someKernel(float *weight, int num_ic, int num_oc, int kw, int kh){
    weight[some nasty pointer math collapsing dimensions into something flat] = ...;
}
// called with
someKernel<<<grid_dim, block_dim>>>(&lenet5.weight0_1, num_ic, num_oc, kernel_width,
kernel_height);

def someKernel(lenet5: LeNet5GPU):
    lenet5.weight0_1[easy, simple, lovely] = ...
    # if i need num_ic, num_oc, and they aren't just a comptime / #define global we
can directly access,
    # i can just do lenet5.weight0_1.shape[0]() to get the 0th dim, shape[1]() for
the next,
    # all I really need to know is the rank. no explicit passing needed.
    # the type contains this and is available!
# called with
someKernel(lenet5, grid_dim = (...), block_dim = (...))
```

Nice. Very nice. How do we do this? We implement `DevicePassable` (which many things do already, allowing lovely types like `LayoutTensors` to be used natively within kernels). Let’s take a look at how to do that.

```
struct MyExample(DevicePassable):
    """This trait marks types as passable to accelerator devices."""
    """We just need to implement the following..."""

    comptime device_type: Self # no changes needed
    # our struct members need to be DevicePassable, lets say we have some data:
    var weight: LayoutTensor[...]

    @staticmethod
    def _is_convertible_to_device_type[SrcT: AnyType]() -> Bool:
```

```

    # has a default implementation, no need to worry about this

def _to_device_type(
    self, mut encoder: Some[DeviceTypeEncoder], target: MutOpaquePointer[_]
):
    encoder.encode(self, target) # simple

@staticmethod
def get_type_name() -> String:
    return reflect[Self].name() # clean, robust "MyExample"

```

That's it! Now any type (assuming all of its members are also DevicePassable) can be used in a sane way. We even got to show some reflection!

However, there's one big thing that we have to tackle: host keeps track of its device allocations by using DeviceBuffers that we need to wrap in LayoutTensors (the address spacing is handled automatically through parameter inferring). DeviceBuffers don't make sense to pass to a device, they're a host-specific construct, so they don't implement DevicePassable. We split those out into their own DeviceBuffers struct, and then the model just wraps around those buffers. The Buffers struct handles allocations / allocations, cleanup, copying from CPU weights, a nice little option to zero out all members between passes.

```

def __init__(out self: LeNet5GPU, bufs: LeNet5GPUBuffers) raises:
    var w0 = bufs.w0l_storage # for some reason we need this step of indirection
    self.weight0_1 = LayoutTensor[ftype, Self.w0_1_layout, MutAnyOrigin](w0)
    ... # and so on

```

That's really it. The last thing to do is that aforementioned Session pattern to tie lifetimes without having to mess with origins manually; see the DeviceSession. It's just the GPU Allocator that we pass to the LeNet5GPUBuffers that we pass to the LeNet5GPU model. Everything stays alive until the compiler detects the last usage of any of those, and only then it will deconstruct. Great - lifetimes are effectively handled, and we gain a clean API!

Features

The feature buffers require more care and attention. This will store the input, the results of each layer, and will be where we collect the outputs. For pure inference, we don't need to store any intermediate results, just enough for whatever current layer we're working on through to the output. A lot of frameworks will do this: only allocate one buffer as large as the largest intermediate would be, and just reuse it over and over for a forward pass. For a backward pass, if you can, it's really nice to just hold on to all of them so you can calculate gradients. I made the decision to just store everything for two main reasons: debugging is easier this way and this model is small enough that I'm really just not worried about "wasting" space.

I do wish I'd made one big change: I'm typically allocating these as an array of structs, where a struct of arrays would be much better. I highly encourage you to do this yourself from the start, it's on my list of things I would change for future implementations. For example:

```

# right now - Array of Structs (AoS)
# Feature has some layers containing shapes like InputLayout = (IN_CHANS, IMAGE_SIZE,
IMAGE_SIZE) etc
var buffers = List[FeatureGPU](capacity = batch_size) # or similar treatment
# fill the list by initializing a FeatureGPU() in each index

# much better for CPU and GPU - Struct of Arrays (SoA) styled
var buffers = FeatureGPU[batch_size]()

```

```
# where batch_size becomes the outermost dimension for each layer
# i.e. InputLayout = (Self.batch_size, IN_CHANS, IMAGE_SIZE, IMAGE_SIZE)

# worst case if we need to run a batch of size < batch_size, we could fill the
remainder with zeros, or just do a drain loop of FeatureGPU[batch_size = 1]() and
route those appropriately. we could even suffer a small AoS like we have now (with
proper routing)
```

Ops

We have our operations / kernels in their own respective files, decoupled from the model. They're more reusable and generic that way, along with being easier to test and read.

CPU

Here, some of our only real obvious options are the following:

- use as much SIMD as possible
- decide if convolutions should implement Winograd (fewer multiplications) or Im2Col (some work for reshaping but we turn convolutions into a matrix multiplication). I decided to save this for a later date and focus on GPU performance. These are also easier with better shapes (SoA).
- we need a good matrix multiplication, there's a BLAS implementation provided by linalg.matmul but that feels a bit like cheating. we'll use it anyways! :)
- kernel fusion - activation functions and biases are very easy to combine
- do we parallelize individual ops to utilize CPU or use CPU threads to handle an image each? with SoA, the answer is certainly the former, and probably gives us better memory access patterns because weights and layers are accessed together instead of potentially diverging based on the OS scheduler

GPU

This is where I spent a good amount of time and could spend a lot more. A lot of the lessons here are things you'd learn from just spending time writing GPU kernels.

To show some of what a kernel actually looks like, let's take a look at our very first kernel. This is going to take in raw pixels from the MNIST repository and prepare them into the feature's input buffers that the model expects. That means we need to normalize and pad our data. Let's walk through this. If you're not super familiar with blocks and threads, any CUDA-centric tutorial should get you up to speed.

```
def normalizeInputsKernel[
    batch_size: Int
](
    raw_pixels: LayoutTensor[
        DType.uint8,
        Layout.row_major(batch_size, IMAGE_SIZE, IMAGE_SIZE), # correct SoA pattern
        is here, at least
        ImmutAnyOrigin,
    ],
    feats: UnsafePointer[FeatureGPU, MutAnyOrigin], # MutAnyOrigin because we need to
    mutate the buffers
):
    """Call with grid_dim=batch_size,
    block_dim=next_power_of_two(IMAGE_SIZE*IMAGE_SIZE) (1D).
    Pads and normalizes raw uint8 pixels into the feature input buffer.
    """
    comptime img_sz = IMAGE_SIZE * IMAGE_SIZE
    comptime reduction_size = next_power_of_two(img_sz) # 28*28 -> 1024.
```

```

var img = block_idx.x
var flat = thread_idx.x
var active = flat < img_sz
# inactive threads clamp to (0,0) – valid index, value masked to 0 below
var row = (flat // IMAGE_SIZE) if active else 0
var col = flat % IMAGE_SIZE
var pix = sftype(rebind[UInt8](raw_pixels[img, row, col])) if active else
sftype(0) # pull in from global memory to local register

# these primitives will handle an efficient reduction for us and place the result
in thread 0.
# if we needed the result in all threads, broadcast=True
var sum_total = block.sum[block_size=reduction_size, broadcast=False](pix)
var sq_total = block.sum[block_size=reduction_size, broadcast=False](pix * pix)

# 2-element shared buffer: [mean, std]. Only thread 0 writes, all active threads
read.
var stats = LayoutTensor[
  ftype, Layout.row_major(2), MutAnyOrigin,
  address_space=AddressSpace.SHARED,
].stack_allocation()
if flat == 0:
  var mean_val = sum_total / sftype(img_sz)
  var temp = sq_total / sftype(img_sz) - mean_val * mean_val
  stats[0] = mean_val
  var temp_fp32 = Float32(temp)
  stats[1] = sftype(sqrt(max(temp_fp32, Float32(0)))) + Float32(1e-7)) # NVIDIA
GPU doesn't support fp64 sqrt (yet?)
  barrier() # make sure when other warps go to read from shared, the result is
indeed ready to be read

if active:
  # buffers are zeroed at arena / allocator init, so padding border is already
0
  feats[img].input[0, row + PADDING, col + PADDING] = (pix - stats[0]) /
stats[1]

```

This shows a number of features: shared memory, block primitives, natural indexing (thank you DevicePassable), and some limitations as well that we have to work around (NVidia doesn't have sqrt supported for fp64, so we have to coerce into a supported dtype). Nice.